



Learn to code, learn to think, and learn to solve

01

Programming & Problem Solving

What is Coding/Programming

Coding is the process of giving instructions to a computer to perform a task

Example: Finding the Largest Number

Input: 12 45 23 67 3

↓
Compare the numbers

↓
Largest = 67



Programming Languages - What should I Learn

Language	Mostly used for
C	Fundamentals, systems and understanding how programming works
C++	A strong choice for DSA and problem solving
Java	A very good alternative for programming and development
Python	Very beginner-friendly and widely used in AI/Data Science

What Actually Happens When you Get a Problem

Don't Immediately code

Understand the problem → Think about then approach

Write the logic/algorithm → Code the solution

Test it with examples

Debug and improve

Think Before You Code!

2, 3, 7, 22, 89, ?



Answer: 446

Pattern:

$$\times 1 + 1$$

$$\times 2 + 1$$

$$\times 3 + 1$$

$$\times 4 + 1$$

$$\times 5 + 1$$

Why DSA?

Dreaming of getting into top companies? 🚀

DSA helps you develop:

- 🧠 Logical Thinking — Understand how to approach a problem
- 🧩 Problem-Solving — Break complex problems into smaller steps
- ⚡ Efficient Solutions — Find better ways to solve problems
- 📁 Technical Interviews — Important foundation for coding interviews

DSA is one of the important foundations for technical interviews

DSA is not just about memorizing data structures
It is about learning how to solve problems effectively



Basic DSA Examples



Array → Student Marks



Stack → Plates



Queue → Ticket Line



Searching → Find a Student



Sorting → Arrange Marks

Advice:

Striver's A2Z DSA sheet link:

Competitive Programming

Solving programming problems in a competitive environment, often with time limits

It helps you improve:

-  Problem-solving
-  Speed & efficiency
-  Logical thinking
-  Coding skills
- Solving programming problems through contests

Coding Practice Platforms

Platforms to Explore :

LeetCode:

→ Practice problems & build problem-solving skills

CodeChef

→ Practice + beginner-friendly contests

Codeforces

→ Explore competitive programming when you're ready

HackerRank

→ Coding practice, problem solving, and skill-building challenges

GeeksforGeeks

→ DSA concepts, tutorials, practice problems, and interview preparation

Where Do I Code?

On Your Laptop

- VS Code — popular code editor
- Write, run, test, and debug your programs
- Good choice as you learn programming

VS Code Setup:

[link:](#)

Online IDEs

- Code directly in your browser
- No complicated installation
- Useful for quick practice and experiments

You don't need a perfect setup

You need a place to write code and the willingness to practice

What Freshers SHOULD NOT Do

- Don't try to learn everything at once
- Don't learn multiple languages simultaneously
- Don't spend weeks only watching tutorials
- Don't copy solutions without understanding them
- Don't avoid practicing by yourself

What Freshers SHOULD Do

- Pick one programming language
- Learn the fundamentals properly
- Practice small problems regularly
- Learn DSA step by step
- Solve problems yourself
- Don't just watch tutorials
- Understand solutions before using them
- Debug your own code
- Practice consistently

Guide to Frontend, Backend, Databases, Git & Deployment

02

Web Development

Every Platform Rests on Three Pillars

Netflix, WhatsApp, ChatGPT, YouTube — all built the same way

1. Frontend

What you see and interact with like pages, buttons, forms.

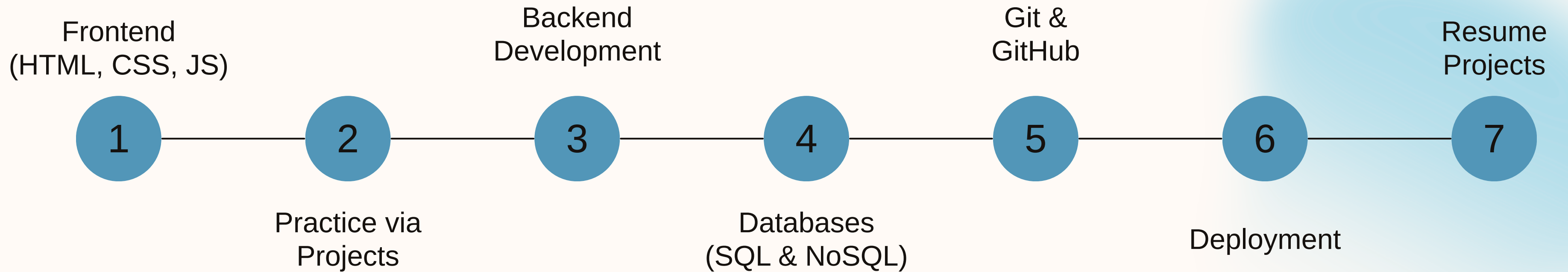
2. Backend

What happens behind the scenes like logic, checks, processing.

3. Database

Where information is stored like accounts, orders, history.

The Learning Path, Step by Step



Frontend Development

Three technologies every developer must know



HTML

HyperText Markup Language

Defines the content and structure like headings, paragraphs, buttons, forms, images.



CSS

Cascading Style Sheets

Adds styling like colors, layout, spacing. Good CSS is what makes a UI feel polished.



JavaScript

Logic & interactivity

Powers what happens on clicks, form submits, and navigation. Brings the site to life.

Backend Development

Difficulty increases here — many languages and stacks to choose from

Language Options

Node.js

JavaScript runtime — reuses what you already know from frontend

Python

Popular, readable, widely used for backend and beyond

Java & others

Also common in industry backend systems

Popular Tech Stacks

- **MERN Stack**
 - MongoDB, Express, React, Node
- **MEAN Stack**
 - MongoDB, Express, Angular, Node
- **LAMP Stack**
 - Linux, Apache, MySQL, PHP

Databases

Relational Databases

Data stored in structured tables. Should be learned properly and in depth

Flexible Storage

Data isn't necessarily in tables. MongoDB is a popular example.

Learn both — different companies use different technologies, and you shouldn't limit your opportunities.

Git - Version Control

Think of it like a bank account — a complete transaction history of your code

Tracking History

V1 - Home Page

V2 - Product Page

V3 - Add to cart

V4 - Payment Feature (buggy)

Roll back to Version 3 when Version 4 breaks — no manual cleanup needed.

Collaboration

In a company, many developers work on the same project at once

- One developer works on login
- Another builds the dashboard
- Git tracks who changed what and prevents work from being overwritten

GitHub — Your Developer Portfolio

Git runs on your computer. GitHub stores and showcases your work online.



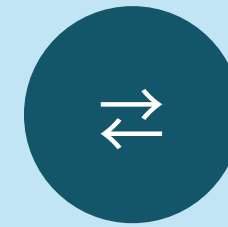
Repositories

Projects live in “repos”
upload, track, and
manage your code
online.



Portfolio

Recruiters check your
GitHub to verify real
project work and
contributions.



Open Source

Explore public projects,
fix bugs, add features,
and collaborate globally.

Deployment

Make your project accessible on the internet — not just your laptop

Free, beginner-friendly platforms:

- GitHub Pages
- Railway
- Render
- Vercel
- Netlify



FROM PROMPTS TO POSSIBILITIES.

03

AI & Career



WEB 1.0

The Read Web

- Websites published information.
- Users mainly read/consumed content.
- Very little interaction or user-generated content.
- Websites were more like digital newspapers.



WEB 2.0

Interactive Web

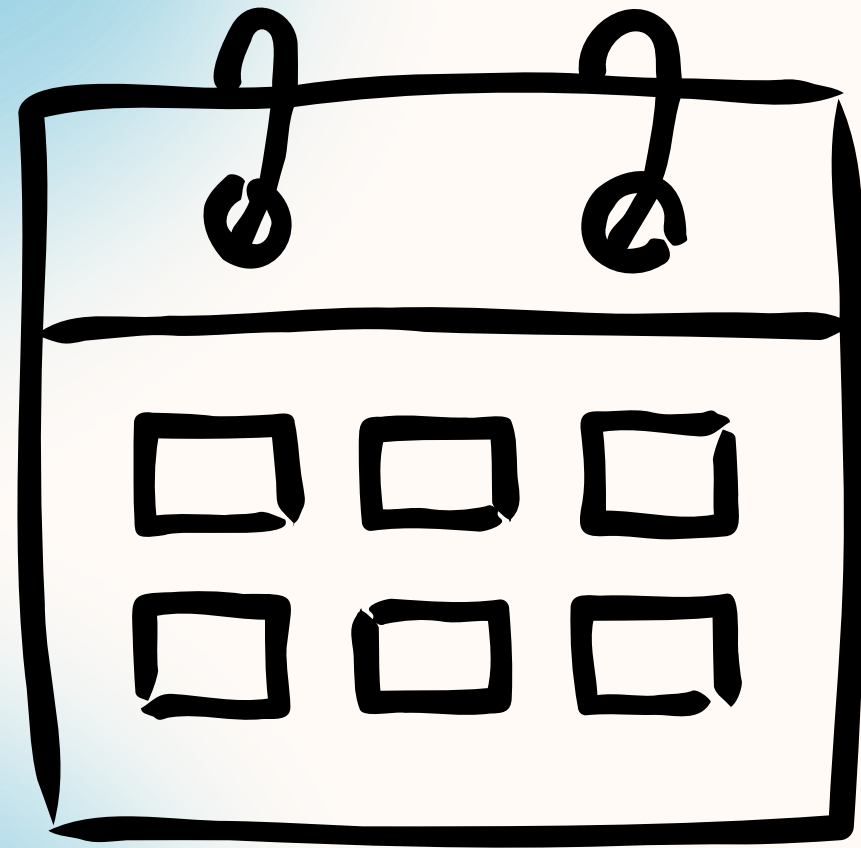
- Users can create and upload content.
- Social media emerged.
- Comments, likes, sharing and collaboration became normal.
- Web apps became much more powerful.



WEB 3.0

Decentralized Web

- More control — users have more control over their digital identity and assets.
- Digital ownership — things you own digitally can be represented and transferred.
- Less dependence on one platform — some services can work without one central company controlling everything.



NOW

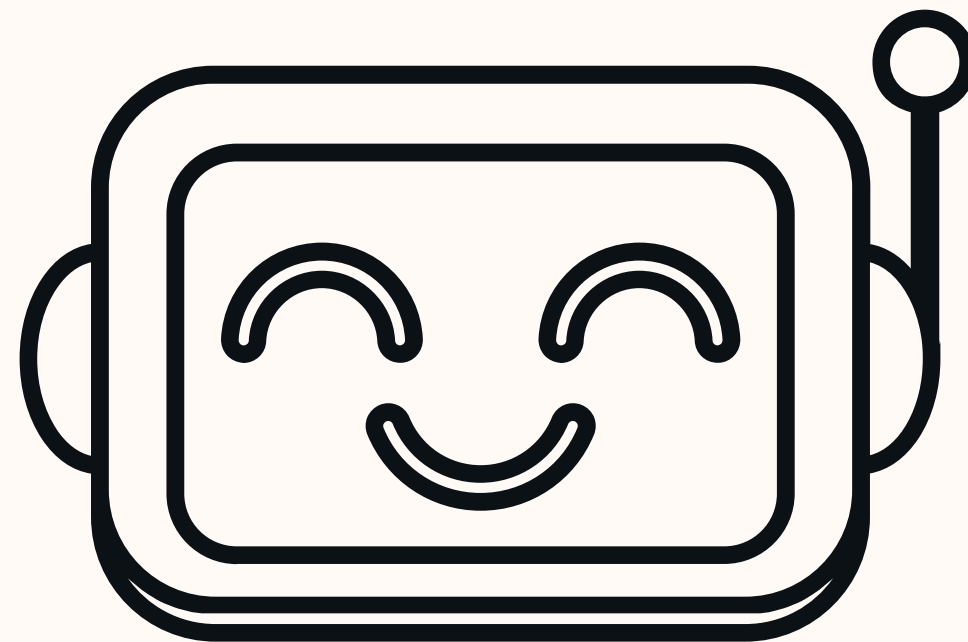
WEB 4.0

Intelligent & autonomous Web

- AI-Driven Intelligence — Systems understand, learn, and adapt.
- Autonomous Automation — AI agents perform tasks with minimal human input.
- Connected Ecosystems — Devices, people, and services work together seamlessly.
- Ambient Computing — Technology becomes invisible, context-aware, and always available.

Do you think, you know AI ?

Have you ever used AI ?





WILL AI REPLACE US?



Let's Not Debate

Let's see what AI can do now

ONE SMALL PROBLEM

IDEA RESEARCH

PRODUCT DEVELOPMENT

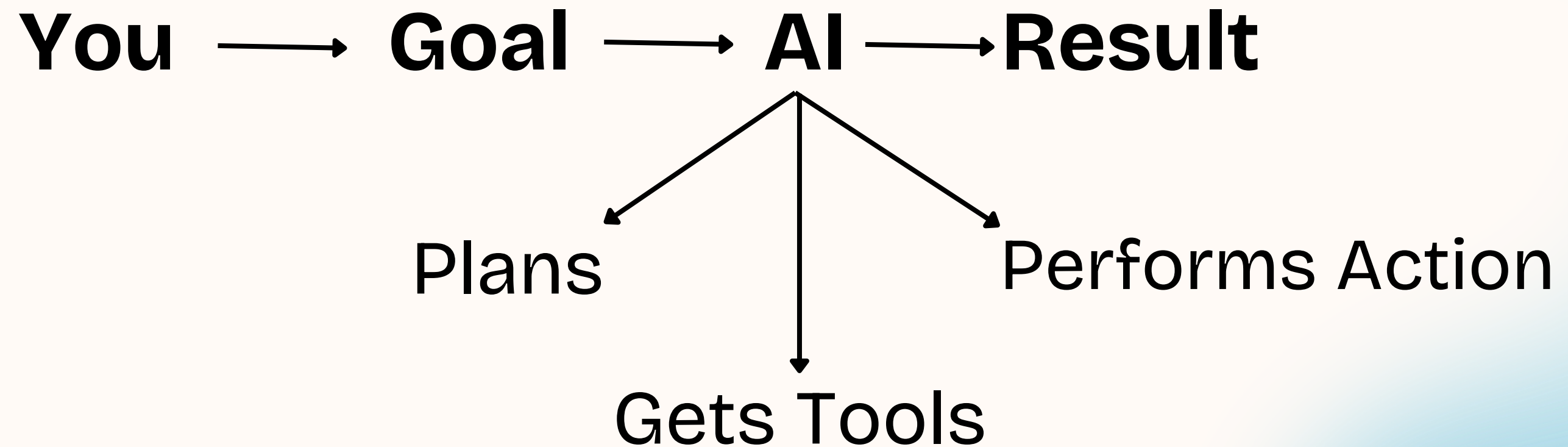
DESIGN(Image)

MARKETING(Video)

PITCH DECK (PRESENTATION)

What if one prompt could do it all?

Agents



**“Find me 20 internships suitable for a first-year
computer science student.”**

SEARCH -> CHECK -> ELIGIBILITY -> COLLECT INFORMATION -> COMPARE

|

RANKS

|

PUT THEM IN A SPREADSHEET

|

DRAFT APPLICATION MATERIAL

API TOKENS.

CREDITS.

BUT THERE'S A CATCH

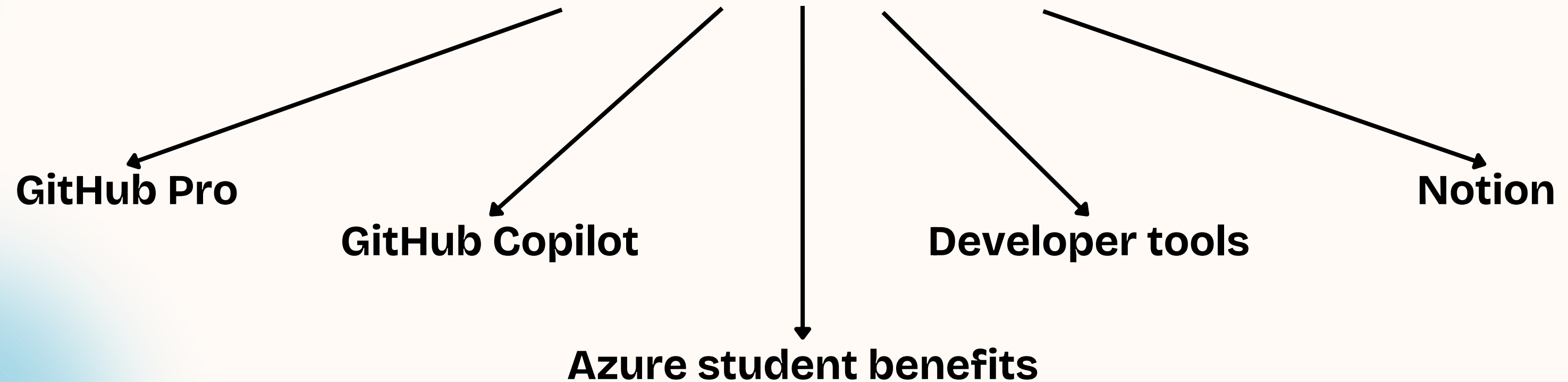
PAID MODELS.

USAGE LIMITS.

API LIMITS.

Here Comes, **THE STUDENT ADVANTAGE**

GITHUB STUDENT DEVELOPER PACK



Tools

LEARN / THINK



ChatGPT



Claude



Gemini

STUDY / RESEARCH



NotebookLM



Perplexity

DEVELOP



GitHub



GitHub
Copilot



Google AI
Studio



Google
Antigravity



Cursor



Replit



Lovable



Bolt

CREATE IMAGES



Gemini



ChatGPT



Microsoft
Designer



Adobe
Firefly



Ideogram



Leonardo AI

CREATE VIDEO



Google
Flow



Kling AI



Runway



Pika



Hailuo AI



Luma Dream
Machine

PRESENT



Gamma



Canva



Beautiful.ai



PowerPoint +
Copilot

EXPERIMENT WITH MODELS



OpenRouter



Google AI
Studio

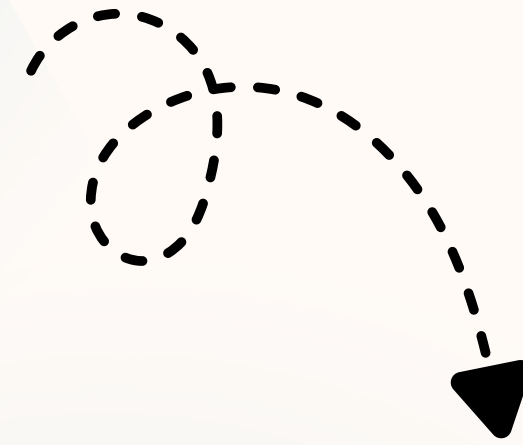
FIND AI TOOLS



There's An AI
For That



Don't memorize
every AI tool



Coming Back

Will AI Replace Us?

But here's the other side.

Uber gave engineers broad access to Claude Code. By **April** 2026, it had already exhausted its annual AI budget because adoption happened much faster than expected

AI Engineer. AI Product Manager. AI Automation Engineer. AI Agent Developer.

AI Safety and Evaluation roles

What should we be good at?

AI → code, desing, research, generate

Thinking • Communication • Creativity •
Leadership • Problem solving • Collaboration •
Judgement • People

where do you actually develop these?

Hackthons

Projects

Events

People Connections

Clubs



EXPLORE . DIVE IN . CONQUER

